COMPRESSED THEOREM SEARCH UNDER PROMPT CONSTRAINTS: A CHEATSHEET-GUIDED SYSTEM FOR EQUATIONAL IMPLICATION OVER MAGMAS

ANONYMOUS WORKING DRAFT

Abstract. We study prompt-only reasoning for equational implication over magmas under the severe interface constraints of SAIR Equational Theories Stage 1. In this task, the submission artifact is not code or weights but a single plain-text prompt of at most 10,240 bytes. The prompt is rendered directly as the complete model input, only two placeholders are substituted, verdict extraction is strict, and unparseable answers are scored as wrong. These rules convert a mathematical reasoning problem into a compression problem: a successful submission must encode a reliable proof-search policy in natural language, within a fixed byte budget, for models that are powerful but execution-fragile.

Our case study is the v28d cheatsheet, a prompt program that defaults to TRUE and outputs FALSE only when a structural invariant or a guarded finite-magma witness proves separation. The mathematical core consists of six structural tests, a spine-depth divisibility check, and four small magma witnesses. On local GPT-OSS-120B evaluations, v28d achieves 96.7% accuracy on a normal 30-problem gate, 83.3% on a hard3 30-problem rotation, 50.0% on a competition-hard 30-problem rotation, and 43.3% on a hard2 30-problem rotation, with 100% parse rate and perfect TRUE recall on the normal and hard3 slices. More important than the scores alone, however, is the failure analysis across the version lineage. We find that mathematically plausible heuristics often fail because they exceed the model's execution capacity. Ungated lookup-table magmas, multi-stage shortcuts, and overgrown test suites can be algebraically sound yet operationally harmful.

The resulting picture is neither "LLMs discover proofs" nor "formal methods solve everything." Instead, we argue for a middle layer: compressed theorem search under prompt constraints. The core design principle is to optimize not only separation power but soundness per byte and executability per token. This viewpoint yields several original methodological ideas: guard design for finite witnesses, asymmetric optimization for TRUE recall, and a distinction between mathematical soundness and model-executability soundness. We present these ideas as a reproducible engineering and mathematical synthesis rooted in the Equational Theories Project and the SAIR competition setting.

1. Introduction

The implication relation between equations over magmas is a deceptively rich mathematical object. A magma is merely a set equipped with one binary operation and no additional axioms. Even with this minimal structure, determining whether one equation implies another over all magmas leads quickly to a large and intricate implication graph. The Equational Theories Project has made this space concrete and explorable by enumerating 4,694 single-equation laws together with a large implication and anti-implication database, Lean-verified proofs, and finite-magma witnesses [1]. The raw search space contains 22,028,942 ordered

Date: April 2026.

implication pairs, far beyond what one would want to handle by ad hoc human inspection alone.

The SAIR Stage 1 challenge imposes an unusual and consequential interface on this underlying mathematics. Instead of submitting a prover, a decision procedure, or a trained model, participants submit a single plain-text cheatsheet of at most 10,240 bytes. The cheatsheet is the full prompt seen by the evaluation model. It may contain instructions, examples, heuristics, and decision procedures, but it may not call tools, fetch data, or rely on external state. Only the placeholders `{{equation1}}` and `{{equation2}}` are substituted. Answers must not only be mathematically good; they must also survive a strict verdict parser and the failure modes of a large language model reasoning under a long, compressed instruction sequence [4, 5].

This combination of universal algebra and prompt compression creates a distinctive scientific problem. A cheatsheet is not just a prompt. It is closer to a compressed proof-search program written in natural language for an unreliable interpreter. The natural language must carry mathematical content, but also control flow, format stability, and operational safety. It must tell the model which tests to run, in what order, when to stop, and when not to trust a tempting but unsafe witness. The fundamental objective is not simply "maximize theorem-proving power." It is to maximize theorem-proving power subject to execution risk, parse risk, and byte budget.

The system documented in this paper, culminating in the v28d prompt, emerged from that design pressure. The core idea is conceptually simple: assume TRUE by default, and only say FALSE when a test establishes a separation of Equation 1 from Equation 2. The tests themselves are chosen from a small menu of structural invariants and tiny finite magmas. The structural tests are cheap, local, and operationally reliable. The finite magmas are more expressive but more execution-sensitive, so they must be carefully ordered and guarded. This design is not equivalent to a complete decision procedure. Rather, it is a deliberately asymmetric recognition system: prioritize safety on TRUE pairs, and attack FALSE pairs using witnesses that are mathematically meaningful but simple enough for a language model to execute.

The empirical results are best interpreted as a case study rather than a universal claim of state-of-the-art performance. On local GPT-OSS-120B evaluations, v28d reaches 96.7% accuracy on a normal gate, 83.3% on hard3, 50.0% on a competition-hard slice, and 43.3% on hard2. These numbers matter, but the more durable contribution lies in the design lessons extracted from the version lineage. Across the repo history, one repeatedly sees mathematically correct ideas fail in practice because the model cannot execute them reliably. Ungated multi-element magmas destroy TRUE recall. Clever shortcuts collapse. Too many marginal tests exhaust the attention budget. Conversely, a smaller number of crisp structural invariants and operationally simple algebraic witnesses can outperform more ambitious but brittle designs.

This paper advances four main claims. First, prompt-only mathematical reasoning under strict size limits should be understood as compressed theorem search, not generic chain-of-thought. Second, in this domain, the mathematics that matters most is not necessarily the most sophisticated mathematics, but the mathematics that balances algebraic validity

with execution stability. Third, small finite magmas are valuable not merely as external countermodels but as prompt-embedded witnesses, provided they are guarded against false separations. Fourth, failure analysis is mathematically informative: it reveals an important distinction between formal soundness and model-executability soundness. A rule may be correct in algebra and still be a bad prompt instruction.

2. Background, Data, and Related Context

An equational law over magmas is an identity between two binary terms built from variables and one binary operation. Typical examples include the trivial law $x = x$, the singleton law $x = y$, commutativity $x \diamond y = y \diamond x$, and associativity $x \diamond (y \diamond z) = (x \diamond y) \diamond z$. For two equations E_1 and E_2, the implication $E_1 \Rightarrow E_2$ means that every magma satisfying E_1 also satisfies E_2. This implication relation organizes equations into a partially ordered structure with equivalence classes, cover relations, and a large body of known and conjectured results.

The Equational Theories Project provides the modern reference environment for this space [1]. The project's stated goal is to map the relations between different equational theories of magmas. It combines human theorem proving, machine support, Lean formalization, proof metadata, and interactive web tooling. The project website exposes an Equation Explorer, a Finite Magma Explorer, a dashboard, documentation, and a downloadable implication graph. The GitHub repository includes equation lists, dual maps, smallest finite magma witnesses, proof metadata, scripts for implication extraction, and a substantial paper and blueprint ecosystem.

For the present work, the project is important in two ways. First, it supplies the mathematical universe in which the SAIR task lives. Second, it shapes the style of reasoning that the cheatsheet can plausibly compress. On a refresh performed for this manuscript on 2026-04-17, the local pipeline recovered the full Teorth core snapshot at upstream commit `d69afe6eee96801b1cbfad2bfca18eb48a74fc2e`. The refreshed assets contained 4,694 equations, a decoded 4694×4694 implication matrix with 1,415 equivalence classes, and 12,373 records in `full_entries.json`. These are the primary external mathematical assets used by the manuscript.

The SAIR Stage 1 challenge uses this broader ecosystem but changes the optimization target. The goal is not to extend the implication database directly, nor to produce Lean proofs, nor to publish new universal algebra theorems in the first instance. Instead, the challenge asks competitors to distill enough reasoning into a short prompt that a fixed set of frontier models can answer held-out implication questions. Official evaluation is prompt-only, no-tools, offline, and based on three models of equal weight: GPT-OSS-120B, Llama 3.3 70B Instruct, and Gemma 4 31B IT [5]. The task is binary classification. Unparseable outputs count as wrong. The evaluation set is balanced 50% TRUE and 50% FALSE, but public hard benchmarks are skewed toward FALSE, which creates a known tendency for weak prompts to drift toward trivial FALSE defaults.

The present paper also has a modest connection to older and larger automated reasoning traditions. From Knuth-Bendix completion we inherit the idea that local rewrite structure can be computationally decisive [2]. From equality saturation and its guided variants we

inherit the idea that dense rewrite spaces can still be navigated effectively when the search representation is well chosen [6, 3]. What changes in the SAIR setting is that the search procedure itself must be encoded as compressed natural language. This makes prompt structure and mathematical structure inseparable.

3. Competition Rules as Design Constraints

The competition rules are not incidental. They define the feasible hypothesis class. The most important constraints are as follows.

First, the submission artifact is one plain-text file of at most 10,240 bytes. This is a hard on-disk budget, not a soft token preference. In the local repo, byte measurement is treated in a CRLF-aware, on-disk sense because that is what matters operationally for the artifact. Second, the cheatsheet is the entire prompt. There is no hidden system prompt wrapping the prompt into a richer scaffold. Third, only two substitutions are allowed: `{{equation1}}` and `{{equation2}}`. Fourth, the model has no browser or tool access during evaluation. Fifth, verdict extraction follows a strict three-tier priority: boxed verdicts first, labeled verdicts second, bare-line verdicts third, with the last occurrence winning within a tier [5]. A mathematically correct argument with an unparsable conclusion is therefore scored as a failure.

This matters for mathematical system design. A conventional theorem prover can separate correctness from presentation. A prompt-only competition cannot. Formatting is part of the semantics. So is refusal to overrun the parser. So is the decision to avoid clever but opaque abbreviations that the model may garble. The cheatsheet must jointly solve a mathematical problem and an output-interface problem.

The three official models also matter. They are not identical interpreters. Historically, different prompt versions in this repo showed large model-specific variation. Some prompts that were coherent on one model failed catastrophically on another. As a result, the local design philosophy gradually shifted toward a more conservative core: fewer tests, clearer gates, less hidden branching, more direct examples, and stronger emphasis on output stability. The final v28d prompt was tested primarily on GPT-OSS-120B, which is a limitation discussed later, but the entire version lineage was shaped by the broader multi-model threat model.

The rules also imply a kind of algorithmic asymmetry. If FALSE answers require witnesses while TRUE answers do not, then one can bias the system toward default TRUE and only incur risk when there is a positive reason to separate. This is not a universal theorem about all tasks. It is specific to a setting where false separation is more harmful than missed negative evidence, where parse stability matters, and where execution-heavy tests can destroy recall on true implications. The rule environment therefore rewards risk-sensitive proof search.

4. Method: Prompt Programs as Compressed Theorem Search

The v28d cheatsheet is best understood as a prompt program. It has a task declaration, a default action, a decision tree, a strict output format, and worked examples. It does

not contain executable code, yet it is meant to induce code-like behavior in the model. The program begins by placing the problem in the standard implication form. It then sets `DEFAULT: TRUE` and instructs the model to say FALSE only when a specific test proves separation. This single line encodes the system's central risk policy.

The rest of the prompt is organized as a sequence of tests. The model first splits each equation around the equality sign. It then applies six structural tests. After each test it consults a decision table with exactly one separating row: $E_1 = \text{HOLD}$ and $E_2 = \text{FAIL}$. If that row occurs, the system stops and returns FALSE. Otherwise it continues. If no structural separation is found, the prompt permits a spine-depth check and then a finite list of small magma witnesses. The output format at the end is rigid and easy to parse.

There are several reasons to call this a compressed decision procedure rather than a loose heuristic prompt. The tests are explicit and ordered. The separation condition is explicit. The halting condition is explicit. The prompt forbids unsupported FALSE answers. The examples are not flavor text; they are operational traces. They train the model's interpreter, not just its verbal style.

Compression pressure changes what counts as a good mathematical device. In a traditional proof system, one might prefer the strongest lemma or the most general countermodel. In a prompt program, one prefers the device that is easiest to state, easiest to execute, hardest to misread, and most robust against transcription errors. That preference shaped nearly every successful choice in the version history.

4.1. **Structural front end.** The question "what mathematics was used the most?" has two different answers. One answer is historical: which ideas survived repeated prompt evolution and kept delivering value. The other is operational: which ideas are actually carrying the classification burden in the final system. In both senses, the answer is the same. The dominant mathematics is not exotic universal algebra. It is a carefully chosen layer of structural invariants, followed by a tiny set of finite witnesses.

The first two tests in v28d are LP and RP: compare the first variable on the left and right sides, and compare the last variable on the left and right sides. These tests look almost trivial, but they are fundamental. They are cheap to compute, naturally aligned with the syntax of binary terms, and easy for a language model to execute. They survive because they give high value per byte.

The third and fourth tests, C0 and VARS, refine the same intuition. C0 asks only whether each side contains the binary operation symbol. It is deliberately coarse. VARS asks whether the variable support on each side matches, with a special case for bare-variable equations. Both tests exploit low-cost syntactic facts that often correspond to genuine algebraic obstructions. Their usefulness was visible early in the project and remained stable through multiple prompt generations.

The fifth and sixth structural tests, COUNT2 and LDEPTH, are more delicate. Early versions of the project repeatedly found that parity-like tests could be mathematically attractive but operationally dangerous. A model that miscounts introduces false separations. COUNT2 survives in v28d not because parity is always safe, but because it is carefully constrained:

TABLE 1. Core structural tests in v28d and their operational role.

Test	Mathematical idea	Why it survived prompt compression
LP	Leftmost variable invariant	Cheap to compute; high value per byte; easy to exemplify
RP	Rightmost variable invariant	Symmetric companion to LP; strong on easy FALSE pairs
C0	Presence of the binary operation	Coarse but robust separator for bare-variable versus compound-term mismatch
VARS	Variable-support comparison	Captures support mismatch without deeper algebra
COUNT2	Parity of variable multiplicities mod 2	Keeps only the simplest stable part of counting logic
LDEPTH	Left-spine depth parity	Replaces brittle depth-mod-3 experiments with a simpler parity test
Spine divisibility	Divisibility of pure left-spine depths	Adds power with little execution burden

counts are taken modulo 2, the examples are concrete, the decision rule remains one-sided, and the rest of the prompt is already organized around conservative control flow. LDEPTH measures the parity of left-spine depth. Earlier depth-mod-3 tests and other more ambitious depth heuristics performed poorly. The successful form is the simplest one that still captures useful algebraic structure: follow the left spine, count binary-operation nodes, and reduce modulo 2.

After the six structural tests, v28d applies a spine-depth divisibility check to a highly restricted shape class: both equations must present as pure left spines and must begin with the left-hand-side variable. If these conditions hold, then the system compares the two depths and asks whether one divides the other. This test adds genuine mathematical power at very low execution cost. There is no lookup table, no branching arithmetic, and no large witness. The model only counts parentheses and compares divisibility.

4.2. **Guarded finite-magma witnesses.** The structural layer alone is not enough. Earlier structural-only systems reached a ceiling on hard FALSE cases. The second major mathematical layer is therefore a small family of 3-element magma witnesses.

T3R uses the rule $a*b = \mathrm{next}(b)$, where `next` cycles $0 \to 1 \to 2 \to 0$. T3L uses $a*b = \mathrm{next}(a)$. T5B uses a table equivalent to $a + 2b \pmod 3$. NL1 uses a fixed nonlinear 3×3 table. These witnesses are not meant to approximate the entire space of finite magmas. They are chosen because they are small, interpretable, and reusable across many pairs.

The key design lesson is that a witness must be judged both algebraically and operationally. T3L is a good example of an operationally strong witness. It is simple to state, easy to

TABLE 2. Finite witness layer in v28d.

Witness	Operation law	Strength	Operational status
T3R	$a * b = \text{next}(b)$	Good separator on many small cases	Useful but empirically unsafe on some 4-variable patterns
T3L	$a * b = \text{next}(a)$	Strong and simple witness	Best behaved rescue witness in the current lineage
T5B	$a * b = a + 2b \pmod 3$	Stronger separator than T3R/T3L on some pairs	Requires an all-ones guard to avoid false separation
NL1	Fixed nonlinear 3×3 table	Additional separation power	Must remain late in the chain and interpreted conservatively

compute, and empirically sound in the repo's internal analysis. T3R is more subtle. It has real separating power, but it also exhibited unsoundness on some four-variable equations due to variable recycling under the 0-1-2 assignment scheme. T5B is stronger as a separator but required an all-ones guard after an unsoundness analysis showed that a single assignment was insufficient. NL1 likewise has value but must be handled carefully.

This leads to one of the paper's central methodological claims: a prompt-embedded counter-model witness should be treated as a guarded test, not a free oracle. The witness is allowed to separate only after the prompt has verified that E_1 genuinely holds under the relevant assignments. The all-zeros and all-ones guards are therefore not cosmetic. They are part of the soundness envelope.

4.3. **Original methodological ideas.** This paper does not claim new universal-algebra theorems. Its originality lies in methodology. Four ideas emerged from the work and appear publishable as systems-and-math contributions.

First, prompt design under a hard byte budget should be optimized by *soundness per byte*, not merely raw coverage. A mathematically sophisticated test may be too expensive if its instructions and examples displace more reliable tests.

Second, prompt-level algebra should be *executability-aware*. Rules like `next(a)` and `next(b)` are especially effective because they can be described procedurally without a full lookup table. Arbitrary 3×3 or 5×5 tables often degrade performance even when they are mathematically defensible, because the model loses track of repeated lookups.

Third, the system relies on *asymmetric safety optimization*. Default TRUE with explicit witness-based separation is more than a heuristic. It is a risk-sensitive policy aligned with the structure of the competition, where false separations and parse failures are especially costly.

Table 3. Main local v28d results on GPT-OSS-120B.

Setting	N	Acc.	F1	TRUE Acc.	FALSE Acc.
Normal r23	30	96.7%	96.8%	100.0%	93.3%
Competition hard r23	30	50.0%	59.5%	73.3%	26.7%
Hard3 r24	30	83.3%	85.7%	100.0%	66.7%
Hard2 r24	30	43.3%	51.4%	60.0%	26.7%

Fourth, the project identifies *guarded witnesses* as a middle layer between theorem provers and unguided prompting. These are not full proofs and not raw guesses. They are compact families of algebraic structures embedded in the prompt with explicit validation checks on E_1.

5. Experimental Setup

All results reported here come from the local evaluation loop built around the repo's canonical evaluator and paid-model tooling. The active prompt mode is raw: the cheatsheet is the complete prompt. The main v28d results discussed in this manuscript are GPT-OSS-120B runs, because those are the complete and current v28d measurements available at the time of writing. The official SAIR Stage 1 setup uses GPT-OSS-120B, Llama 3.3 70B Instruct, and Gemma 4 31B IT with temperature 0.0, max output tokens 8192, pinned providers, and strict verdict extraction [5].

The local repo distinguishes several benchmark families. The *normal* slice is used for safety gating. The *hard*, *hard2*, and *hard3* families stress progressively more difficult cases, though they are not interchangeable and should not be collapsed into a single metric. The competition-hard slice used here is a local rotation aligned with the public hard pool, not the official hidden final evaluation set.

The evaluator tracks more than accuracy alone. For each run it records total accuracy, TRUE accuracy, FALSE accuracy, F1, parse rate, elapsed time, token usage, and per-problem raw responses. This matters because a prompt-only system can fail either mathematically or operationally. A drop in parse rate or a collapse in TRUE recall is often more informative than a single aggregate score.

6. Results

Several observations matter.

First, the normal gate result is excellent for a prompt-only artifact. A 96.7% score with 100% TRUE recall means the system almost never rejects a true implication on the normal slice. This is the most important safety result in the repo's current state, because the development policy always treated normal-set safety as the gating criterion for promotion.

Second, the hard3 result is strong and has the same safety signature: 100% TRUE recall with 66.7% FALSE accuracy. This means that on the hard3 slice, the system captured every

TRUE case while still finding a substantial fraction of FALSE separations. In practice, this is evidence that the two-layer design works: the structural tests protect recall, and the guarded witness layer captures a useful share of difficult negatives.

Third, the competition-hard and hard2 results remain far from a full solution. This is important to say plainly. The present paper is not about achieving complete dominance on the hardest public pools. It is about documenting what kind of mathematical compression works under the prompt interface and what kind fails. A 50.0% competition-hard score and a 43.3% hard2 score show that many false implications still evade the available witness family, and some true implications remain vulnerable to execution errors or witness unsoundness.

Fourth, parse rate is 100% in all four runs. This should not be ignored. In this competition, formatting is part of performance. The consistent parse rate indicates that the output template and worked examples are functioning as intended.

The competition-hard result also improved directly over the immediately previous version, v28c. The only intended change between v28c and v28d was a T5B all-ones guard. On the tracked competition-hard slice, accuracy rose from 43.3% to 50.0%, with two previously incorrect T5B-related cases flipping to correct. This is a particularly clean example of a mathematically motivated and operationally successful guard.

7. Failure Analysis Across the Version Lineage

The repo's failures are unusually informative. Unlike a single successful prompt with no history, this project records many broken designs, near misses, regressions, and rollbacks. The failures reveal what kinds of mathematics survive prompt compression.

The earliest class of failure is underpowered simplicity. A pure-magma prompt with no structural tests performed reasonably on tiny tests but lacked the cheap syntactic filters needed to catch obvious FALSE cases. Structural-only prompts performed much better on normal slices but hit a clear ceiling on hard FALSE cases. This established the need for a two-layer system.

The next class of failure is the addition of mathematically valid but operationally brittle tests. Early parity-based XOR variants and mod-3 depth tests often looked appealing. In theory they refined the structural picture. In practice they asked the model to count or recurse in ways that it handled unreliably. The result was false separations on TRUE pairs. The lesson is stark: if a model cannot execute a test consistently, the test is worse than absent, even if the underlying mathematics is fine.

A third class of failure comes from overfitting simplification. Several v23 and v24 variants had a strong structural core and excellent normal performance but no pathway through the hard FALSE ceiling. These versions were not "wrong"; they were incomplete. They demonstrate the limit of structure without finite witnesses.

A fourth class is the near-miss architecture. Versions such as v24i and v26a had the right broad shape: structural pre-screening plus algebraic rescue. Yet they still failed because the rescue instructions were too terse or too unclear. The model had enough information to in

Table 4. Failure patterns that repeatedly appeared in the version lineage.

Failure pattern	Typical cause	Lesson
Too simple	Structure only, no witness layer	Structural reasoning alone hits a hard FALSE ceiling
Wrong tests	Mathematically valid but execution-heavy parity or depth variants	A test the model cannot execute reliably is worse than no test
Near miss	Good architecture, unclear rescue instructions	Interpreter scaffolding matters as much as the witness itself
Kitchen sink	Too many tests, tables, or shortcuts	Attention budget is finite; more tests often mean more mistakes
Ungated multi-element magmas	Arithmetic or lookup logic forced on TRUE pairs	Formal soundness does not imply model-executability soundness

principle execute the witness, but not enough scaffolding to do so reliably. This is a purely prompt-level form of error: the mathematics is present, but the interpreter fails.

The fifth and most revealing class is the kitchen-sink collapse. Several versions added more tests, more tables, more magmas, or more clever shortcuts. The result was usually worse, not better. Ungated multi-element magmas are the clearest example. They are mathematically legitimate and, in external Python evaluators, often perfectly sound. But inside the prompt they force the model into many-step arithmetic or lookup execution on both TRUE and FALSE pairs. TRUE accuracy collapses. This is the central distinction that emerged from the project: mathematical soundness is not the same as model-executability soundness.

The v27 line is the strongest cautionary example. It combined a solid structural core with ungated multi-element magma tests intended as a safety net before returning TRUE. The effect was catastrophic. TRUE accuracy cratered because the model dutifully executed hard arithmetic on TRUE pairs and got the calculations wrong. This is not a story about bad math. It is a story about a prompt system failing to respect the computational habits of its interpreter.

8. What Mathematics Carried the Most of the Work?

The prompt budget forces a mathematical prioritization that is unusual from a traditional universal-algebra perspective. The mathematics that matters most is the mathematics that can be compressed into short, reusable, robust tests.

The first clear lesson is that low-level structural invariants are indispensable. They are not embarrassing approximations of "real mathematics." They are the real mathematics of this prompt regime. They capture genuine constraints on term behavior while staying within the model's reliable execution range.

The second lesson is that finite magmas are extraordinarily valuable, but only when the witness family is small, interpretable, and guarded. The repo's empirical analysis is consistent with the broader Equational Theories Project observation that finite magmas refute a large share of false implications. But prompt-level use of finite magmas is more delicate than offline use. A witness family that is sufficient for programmatic evaluation may still be unusable inside a prompt if the model cannot execute it.

The third lesson is that the distinction between TRUE and FALSE implications should shape the entire architecture. TRUE implications are fragile under overactive heuristics. FALSE implications are fragile under underpowered witnesses. The successful system handles this by having a wide safe front end and a narrow, carefully guarded witness back end.

The fourth lesson is negative. Sophisticated symbolic tricks do not automatically survive language-model interpretation. The repo repeatedly discovered that shortcuts, compressed tables, or too many marginal refinements create confusion rather than power. If a test cannot be explained cleanly enough to be executed by the model under pressure, then from the standpoint of this interface it is not yet a good test.

9. Teorth Refresh and Live Proof-Scrape Notes

For this manuscript we refreshed the Teorth core assets through the local fetch pipeline. The refresh restored previously missing copies of `equations.txt`, `duals.json`, and `smallest_magma.txt`, and re-fetched `graph.json` and `full_entries.json`. The decode checks reported 4,694 equations, a 4694×4694 matrix with 1,415 equivalence classes, and 12,373 metadata records in `full_entries.json`. This makes the manuscript's use of Teorth data explicit, current, and reproducible.

We also ran a bounded live scrape of twelve proof pages through the local `proof_scraping_lab.py` tool. All twelve pages returned successfully from the live website, which confirms that the scrape endpoint was operational at manuscript time. However, the extracted sample was intentionally treated as exploratory. The sample mainly recovered templated page titles and did not surface stable theorem-link structure rich enough to act as a primary scholarly source. For that reason, the manuscript cites the Equational Theories Project itself and its core downloadable assets, while treating the HTML proof scrape as qualitative background only.

This distinction is important for publication hygiene. Upstream project assets such as `equations.txt`, `graph.json`, and `full_entries.json` are citable because they are official project outputs. Local HTML scrape caches are not citable in the same way. They are best regarded as internal exploratory artifacts that can guide narrative interpretation or suggest case studies.

10. Limitations and Threats to Validity

The clearest limitation is model coverage. The current v28d headline results are GPT-OSS-120B runs. The official competition will weight three models equally. Historical versions

such as v24j showed encouraging cross-model normal performance, but v28d itself has not yet received the same level of cross-model validation in this repo. Any claim about v28d as a multi-model artifact should therefore be regarded as provisional.

The second limitation is rotation variance. The repo documents very large variance across some hard3 rotations for earlier versions. The hard3 r24 result reported here is strong, but one should not mistake one rotation for a complete population estimate. The right scientific response is to disclose the variance and, in future work, average across multiple rotations with confidence intervals.

The third limitation is benchmark mismatch. Normal, hard, hard2, hard3, and competition-hard are not interchangeable. They were curated for different roles. A prompt that is excellent on one may not dominate another. This is not noise; it reflects differences in the distribution of separable FALSE pairs and execution-sensitive TRUE pairs.

The fourth limitation is that the system is not a full decision procedure. When the prompt returns TRUE after failing to separate, it is not certifying the implication by proof. It is reporting that the current collection of structural tests and guarded witnesses found no counterexample. This is a design choice aligned with the competition, but the logical interpretation should be stated carefully.

The fifth limitation concerns proof scraping. The sampled live pages confirm endpoint availability, not theorem-level completeness of the scrape parser. Until a richer and more stable theorem-link extraction pass is available, the scrape should remain background evidence only.

11. Conclusion

The SAIR Stage 1 prompt interface forces a clean and difficult question: how much mathematical reasoning about equational implication can be compressed into a single 10 KB prompt and still survive execution by large language models? The v28d system offers one concrete answer. A surprisingly strong and stable core emerges when the prompt is treated as a compressed theorem-search policy rather than as generic chain-of-thought advice. The mathematical devices that endure are crisp structural invariants, simple divisibility checks, and tiny guarded finite witnesses. The devices that fail are usually not bad mathematics, but mathematics that demands too much executional precision from the interpreter.

This leads to a broader thesis. Prompted mathematical reasoning under hard interface constraints should be studied as a joint problem in algebra, compression, and interpreter design. The formal object may be an implication between equations, but the practical object is a natural-language decision procedure executed by a model with known failure modes. In that setting, robustness comes from guard design, asymmetric safety policy, and relentless attention to executability.

The resulting system does not solve the full equational-implication problem. It does, however, identify a viable middle layer between theorem provers and unguided language models. That middle layer is built from compact witness families, structural invariants, and explicit

separation logic. It is mathematically grounded, empirically auditable, and, in the context of SAIR Stage 1, competitive enough to justify deeper study.

Appendix A. Prompt Architecture Summary

The final prompt, v28d, has the following layers.

1. Task framing and default policy.
2. Equation splitting around equality.
3. Six structural tests: LP, RP, C0, VARS, COUNT2, LDEPTH.
4. A four-row decision table with exactly one separating row.
5. A restricted spine-depth divisibility test.
6. Four small magma witnesses: T3R, T3L, T5B, NL1.
7. A rigid parse-safe output format.
8. Worked examples for LP, RP, C0, COUNT2, T3R, and T5B.

The design principle is to spend bytes only where they simultaneously improve mathematical discrimination and operational reliability.

Appendix B. Internal Result Summary

v28d main local results:

- Normal r23: 29/30 correct, 96.7% accuracy, 100% TRUE accuracy, 93.3% FALSE accuracy, parse rate 100%.
- Competition hard r23: 15/30 correct, 50.0% accuracy, 73.3% TRUE accuracy, 26.7% FALSE accuracy, parse rate 100%.
- Hard3 r24: 25/30 correct, 83.3% accuracy, 100% TRUE accuracy, 66.7% FALSE accuracy, parse rate 100%.
- Hard2 r24: 13/30 correct, 43.3% accuracy, 60.0% TRUE accuracy, 26.7% FALSE accuracy, parse rate 100%.

Version-line insight:

- v28c to v28d changed only the T5B guard.
- That change improved the competition-hard slice from 43.3% to 50.0%.
- The normal gate remained safely high at 96.7%.

Appendix C. Reproducibility Notes

The manuscript is anchored to refreshed local evidence and official external sources. The primary local evidence is contained in the committed result JSONs for v28d. The primary external mathematical evidence is contained in the refreshed Teorth snapshot at commit `d69afe6eee96801b1cbfad2bfca18eb48a74fc2e`. The proof-page scrape sample is intentionally auxiliary.

References

[1] Matthew Bolan, Joachim Breitner, Jose Brox, Nicholas Carlini, Mario Carneiro, Floris van Doorn, Martin Dvorak, Andrés Goens, Aaron Hill, Harald Husum, Hernán Ibarra Mejia, Zoltan A. Kocsis, Bruno Le Floch, Amir Livne Bar-on, Lorenzo Luccioli, Douglas McNeil, Alex Meiburg, Pietro Monticone, Pace P. Nielsen, Emmanuel Osalotioman Osazuwa, Giovanni Paolini, Marco Petracci, Bernhard Reinke, David Renshaw, Marcus Rossel, Cody Roux, Jérémy Scanvic, Shreyas Srinivas, Anand Rao Tadipatri, Terence Tao, Vlad Tsyrklevich, Fernando Vaquerizo-Villar, Daniel Weber, and Fan Zheng. The Equational Theories project. GitHub repository, 2024. Accessed 2026-04-17. URL: `https://github.com/teorth/equational_theories`.

[2] Donald E. Knuth and Peter B. Bendix. Simple word problems in universal algebras. In *Computational Problems in Abstract Algebra (Proc. Conf., Oxford, 1967)*, pages 263–297. Pergamon, Oxford-New York-Toronto, Ont., 1970.

[3] Thomas Koehler, Andrés Goens, Siddharth Bhat, Tobias Grosser, Phil Trinder, and Michel Steuwer. Guided equality saturation. *Proc. ACM Program. Lang.*, 8(POPL):1727–1758, 2024. `doi:10.1145/3632900`.

[4] SAIR Foundation. Mathematics distillation challenge: Equational theories stage 1 overview. Competition website, 2026. Accessed 2026-04-17. URL: `https://competition.sair.foundation/competitions/mathematics-distillation-challenge-equational-theories-stage1/overview`.

[5] SAIRcompetition. Stage 1 judge for the mathematics distillation challenge: Equational theories. GitHub repository, 2026. Accessed 2026-04-17. URL: `https://github.com/SAIRcompetition/equational-theories-stage1-judge`.

[6] Max Willsey, Chandrakana Nandi, Yisu Remy Wang, Oliver Flatt, Zachary Tatlock, and Pavel Panchekha. egg: Fast and extensible equality saturation. *Proc. ACM Program. Lang.*, 5(POPL):1–29, 2021. `doi:10.1145/3434304`.